Large Language Model-

The large language models is one of most exciting technological developments.

It has opened up endless possibilities in artificial intelligence, offering solutions to real-world problems across various industries.

One of the fascinating applications of these models is developing custom question-answering or chatbots that draw from personal or organizational data sources.

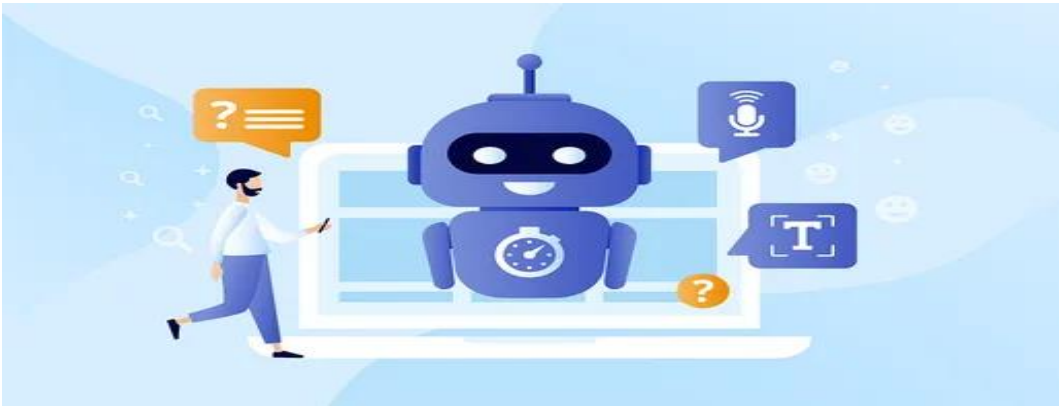
However, since LLMs are trained on general data available publicly, their answers may not always be specific or useful to the end user.

We can use frameworks such as LangChain to solve this issue to develop custom chatbots that provide specific answers based on our data.

Here you will learn how to build custom Q&A applications with deployment on the Streamlit Cloud.

https://www.analyticsvidhya.com/blog/2023/08/qa-applications/

Overview of Q&A Applications



Source: ScienceSoft

Question-answering or “chat over your data” is a widespread use case of LLMs and LangChain. LangChain provides a series of components to load any data sources you can find for your use case. It supports many data sources and transformers to convert into a series of strings to store in vector databases. Once the data is stored in a database, one can query the database using components called retrievers. Moreover, by using LLMs, we can get accurate answers like chatbots without juggling through tons of documents.

LangChain supports the following data sources. As you can see in the image, it allows over 120 integrations to connect every data source you may have.

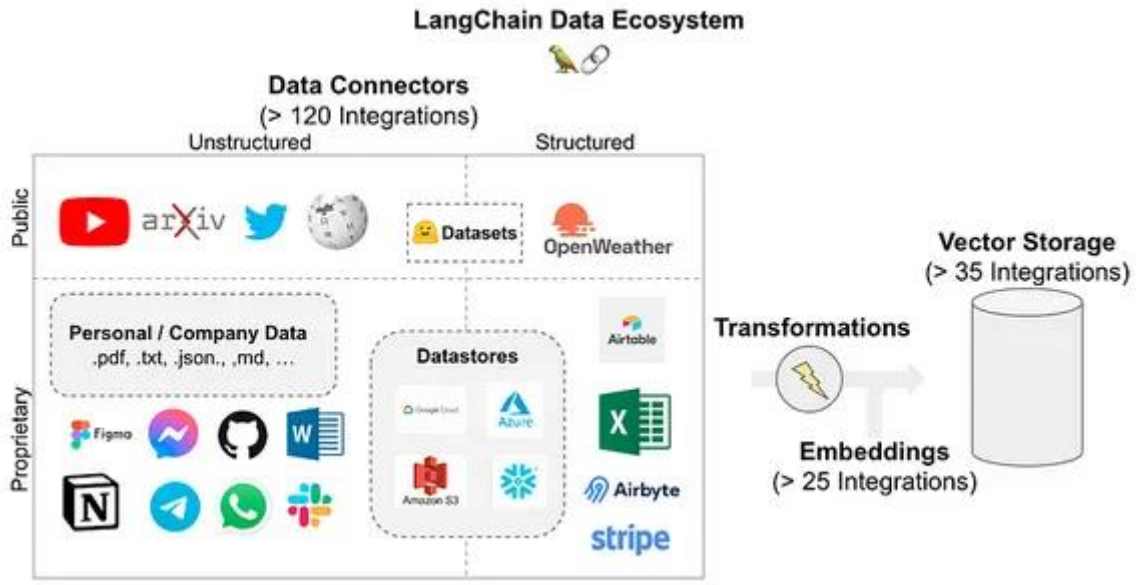


Fig Source: LangChain Docs

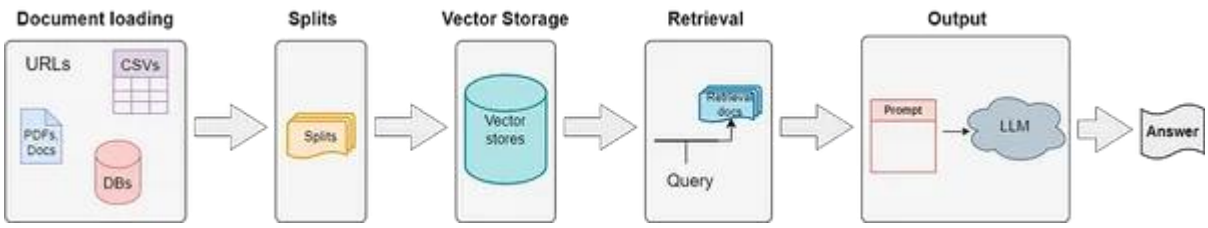
Q&A Applications Workflow

We learned about the data sources supported by LangChain, which allows us to develop a question-answering pipeline using the components available in LangChain. Below are the components used in document loading, storage, retrieval, and generating output by LLM.

- **Document loaders:** To load user documents for vectorization and storage purposes
- **Text splitters:** These are the document transformers that transform documents into fixed chunk lengths to store them efficiently
- **Vector storage:** Vector database integrations to store vector embeddings of the input texts

- **Document retrieval:** To retrieve texts based on user queries to the database. They use similarity search techniques to retrieve the same.
- **Model output:** Final model output to the user query generated from the input prompt of query and retrieved texts.

This is the high-level workflow of the question-answering pipeline, which can solve many real-world problems.



Q&A app – A workflow diagram (Image by Author)

Advantages of Custom Q&A Applications Over a Model Fine-tuning

- Context-specific answers
- Adaptable to new input documents
- No need to fine-tune the model, which saves the cost of model training
- More accurate and specific answers rather than general answers

What is a Pinecone Vector Database?

Pinecone is a popular vector database used in building LLM-powered applications. It is versatile and scalable for high-performance AI applications. It's a fully managed, cloud-native vector database with no infrastructure hassles from users.

LLM bases applications involve large amounts of unstructured data, which require sophisticated long-term memory to retrieve information with maximum accuracy. Generative AI applications rely on semantic search on vector embeddings to return suitable context based on user input.

Pinecone is well suited for such applications and optimized to store and query many vectors with low latency to build user-friendly applications. Let's learn how to create a pinecone vector database for our question-answering application.

Code-

```
  # install pinecone-client
pip install pinecone-client
# import pinecone and initialize with your API key and environment name
import pinecone
pinecone.init(api_key="YOUR_API_KEY", environment="YOUR_ENVIRONMENT")


# create your first index to get started with storing vectors
pinecone.create_index("first_index", dimension=8, metric="cosine")


# Upsert sample data (5 8-dimensional vectors)
index.upsert([
   ("A", [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]),
   ("B", [0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2, 0.2]),
   ("C", [0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3]),
   ("D", [0.4, 0.4, 0.4, 0.4, 0.4, 0.4, 0.4, 0.4]),
   ("E", [0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5])
])
# Use list_indexes() method to call a number of indexes available in db
pinecone.list_indexes()
[Output]>>> ['first_index']
```

In the above demonstration, we install a pinecone client to initialize a vector database in our project environment. Once the vector database is initialized, we can create an index with the required dimension and metric to insert vector embeddings into the vector database.

Building a Semantic Search Pipeline Using OpenAI and Pinecone

We learned that there are 5 steps in the question-answering application workflow. In this section, we will perform the first 4 steps: document loaders, text splitters, vector storage, and document retrieval.

To perform these steps in your local environment or cloud bases notebook environment like Google Colab, you must install some libraries and create an account on OpenAI and Pinecone to obtain their API keys, respectively. Let’s start with the environment setup:

Step 1: Installing Required Libraries

```
# install langchain and openai with other dependencies
!pip install --upgrade langchain openai -q
!pip install pillow==6.2.2
!pip install unstructured -q
!pip install unstructured[local-inference] -q
!pip install detectron2@git+https://github.com/facebookresearch/detectron2.git@v0.6 /
                                #egg=detectron2 -q
!apt-get install poppler-utils
!pip install pinecone-client -q
!pip install tiktoken -q


# setup openai environment
import os
os.environ["OPENAI_API_KEY"] = "YOUR-API-KEY"


# importing libraries
import os
import openai
import pinecone
from langchain.document_loaders import DirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import Pinecone
from langchain.llms import OpenAI
from langchain.chains.question_answering import load_qa_chain
```

After the installation setup, import all the libraries mentioned in the above code snippet. Then, follow the next steps below:

Step 2: Load the Documents

In this step, we will load the documents from the directory as a starting point for the AI project pipeline. we have 2 documents in our directory, which we will load into our project environment.

```
#load the documents from content/data dir

directory = '/content/data'


# load_docs functions to load documents using langchain function

def load_docs(directory):

  loader = DirectoryLoader(directory)

  documents = loader.load()

  return documents


documents = load_docs(directory)

len(documents)

[Output]>>> 5
```

Step 3: Split the Texts Data

Text embeddings and LLMs perform better if each document has a fixed length. Thus, Splitting texts into equal lengths of chunks is necessary for any LLM use case. we will use ***'RecursiveCharacterTextSplitter'*** to convert documents into the same size as text documents.

```
# split the docs using recursive text splitter

def split_docs(documents, chunk_size=200, chunk_overlap=20):

  text_splitter = RecursiveCharacterTextSplitter(chunk_size=chunk_size,
chunk_overlap=chunk_overlap)

  docs = text_splitter.split_documents(documents)

  return docs


# split the docs

docs = split_docs(documents)

print(len(docs))

[Output]>>>12
```

Step 4: Store the Data in Vector Storage

Once the documents are split, we will store their embeddings in the vector database Using OpenAI embeddings.

```
# An example query to our database

query = "What are the different types of pet animals are there?"


# do a similarity search and store the documents in result variable

result = index.similarity_search(

   query,  # our search query

   k=3  # return 3 most relevant docs

)

-

--------------------------------[Output]------------------------------------

result

[Document(page_content='Small mammals like hamsters, guinea pigs,

and rabbits are often chosen for their

low maintenance needs. Birds offer beauty and song,

and reptiles like turtles and lizards can make intriguing pets.',

metadata={'source': '/content/data/Different Types of Pet Animals.txt'}),

 Document(page_content='Pet animals come in all shapes and sizes, each suited

to different lifestyles and home environments. Dogs and cats are the most

common, known for their companionship and unique personalities. Small',

metadata={'source': '/content/data/Different Types of Pet Animals.txt'}),

 Document(page_content='intriguing pets. Even fish, with their calming presence

, can be wonderful pets.',

metadata={'source': '/content/data/Different Types of Pet Animals.txt'})]
```

Step 5: Retrieve Data from the Vector Database

We will retrieve the documents at this stage using a semantic search from our vector database. we have vectors stored in an index called ***"langchain-project"*** and once we query to the same as below, we would get most similar documents from the database.

```
 # An example query to our database
```

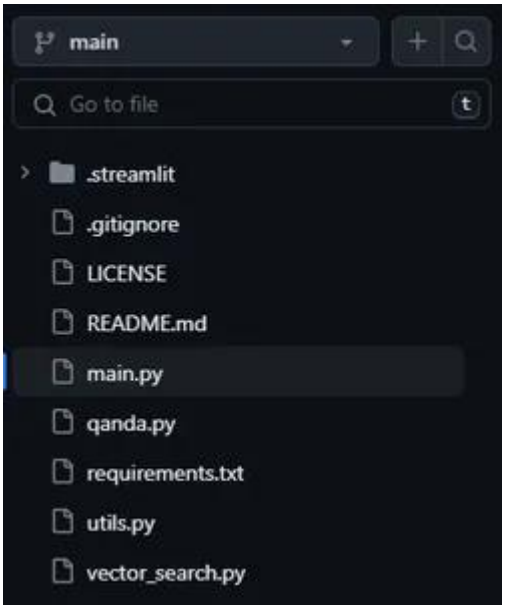
```
query = "What are the different types of pet animals are there?"


# do a similarity search and store the documents in result variable
result = index.similarity_search(
   query,  # our search query
   k=3  # return 3 most relevant docs
)
-
--------------------------------[Output]-----------------------------------
result
[Document(page_content='Small mammals like hamsters, guinea pigs,
and rabbits are often chosen for their
low maintenance needs. Birds offer beauty and song,
and reptiles like turtles and lizards can make intriguing pets.',
metadata={'source': '/content/data/Different Types of Pet Animals.txt'}),
 Document(page_content='Pet animals come in all shapes and sizes, each suited
to different lifestyles and home environments. Dogs and cats are the most
common, known for their companionship and unique personalities. Small',
metadata={'source': '/content/data/Different Types of Pet Animals.txt'}),
 Document(page_content='intriguing pets. Even fish, with their calming presence
, can be wonderful pets.',
metadata={'source': '/content/data/Different Types of Pet Animals.txt'})]
```

We can retrieve the documents based on a similarity search from the vector store, as shown in the above code snippet.

Custom Question Answering Application with Streamlit

In the final stage of the question-answering application, we will integrate every workflow component to build a custom Q&A application that allows users to input various data sources like web-based articles, PDFs, CSVs, etc., to chat with it. thus making them productive in their daily activities. We need to create a GitHub repository and add the following files.



Repo structure

Add these Project Files

- **main.py** — A python file containing streamlit front-end code
- **qanda.py** — Prompt design and Model output function to return an answer to users' query
- **utils.py** — Utility functions to load and split input documents
- **vector_search.py** — Text embeddings and Vector storage function
- **requirements.txt —** Project dependencies to run the application in streamlit public cloud

We are supporting two types of data sources in this project demonstration:

- **Web URL-based text data**
- **Online PDF files**

These two types contain a wide range of text data and are most frequent for many use cases. You can see the main.py python code below to understand the app's user interface.

Code-

```
# import necessary libraries

import streamlit as st

import openai

import qanda

from vector_search import *

from utils import *

from io  import StringIO
```

```
# take openai api key in
api_key = st.sidebar.text_input("Enter your OpenAI API key:", type='password')
# open ai key
openai.api_key = str(api_key)

# header of the app
_ , col2,_ = st.columns([1,7,1])
with col2:
    col2 = st.header("Simplchat: Chat with your data")
    url = False
    query = False
    pdf = False
    data = False
    # select option based on user need
    options = st.selectbox("Select the type of data source",
                           options=['Web URL','PDF','Existing data source'])
    #ask a query based on options of data sources
    if options == 'Web URL':
        url = st.text_input("Enter the URL of the data source")
        query = st.text_input("Enter your query")
        button = st.button("Submit")
    elif options == 'PDF':
        pdf = st.text_input("Enter your PDF link here")
        query = st.text_input("Enter your query")
        button = st.button("Submit")
    elif options == 'Existing data source':
        data= True
        query = st.text_input("Enter your query")
        button = st.button("Submit")
```

```
# write code to get the output based on given query and data sources
if button and url:
    with st.spinner("Updating the database..."):
        corpusData = scrape_text(url)
        encodeaddData(corpusData,url=url,pdf=False)
        st.success("Database Updated")
    with st.spinner("Finding an answer..."):
        title, res = find_k_best_match(query,2)
        context = "\n\n".join(res)
        st.expander("Context").write(context)
        prompt = qanda.prompt(context,query)
        answer = qanda.get_answer(prompt)
        st.success("Answer: "+ answer)

# write a code to get output on given query and data sources
if button and pdf:
    with st.spinner("Updating the database..."):
        corpusData = pdf_text(pdf=pdf)
        encodeaddData(corpusData,pdf=pdf,url=False)
        st.success("Database Updated")
    with st.spinner("Finding an answer..."):
        title, res = find_k_best_match(query,2)
        context = "\n\n".join(res)
        st.expander("Context").write(context)
        prompt = qanda.prompt(context,query)
        answer = qanda.get_answer(prompt)
        st.success("Answer: "+ answer)

if button and data:
    with st.spinner("Finding an answer..."):
        title, res = find_k_best_match(query,2)
```

```
    context = "\n\n".join(res)
    st.expander("Context").write(context)
    prompt = qanda.prompt(context,query)
    answer = qanda.get_answer(prompt)
    st.success("Answer: "+ answer)


# delete the vectors from the database
st.expander("Delete the indexes from the database")
button1 = st.button("Delete the current vectors")
if button1 == True:
   index.delete(deleteAll='true')
```

---

1. https://youtu.be/78JFdQd04Kg
2. https://www.youtube.com/watch?v=_R_d3ibfttY
3. https://docs.langchain.com/oss/python/langchain/voice-agent

To generate a voice agent

2. To Build Langchain application-

(Link - https://www.google.com/search?smstk=Chh0OU5acXlIQ01BU3ExNmNQR1BNejJsST0QARgA&smstidx=1&q=applications+of+langchain&udm=50&csuir=1&aep=34&kgs=1ac87fa5df2ebe4d&shem=agpl,aiolsbe,asdpe,asple,epsd1,rimspwouoe&shndl=37&shmd=H4sIAAAAAAAA_3WOMWoDMRBF2S64CaRLKhF3hqyyjYuQW-QAYlZSRgPSSEiz2S33WD6Dr-TGa3CqkPbDe__tLt1u_yW-NCVZjRNFpxwIqOqlkv-BqAA9y8tnECntQ-t5nntsAkK2tznp5qHa8FZqTlkDJdMCVG8kTGlkoNgXxtenXxpzxuh7IH14hlIi2U2Uuan8rSIw2gDEy_q4dqeHc2f-O6W0VTU9VmBHjHctGqzgaMvVw_JnMxvPDqozw_HdlVvXFR6r92v-AAAA&shmds=v1_ARwrE21kiABMWr7cbXRHX0iCosE8rOQU1zZFGjEU3zt2wPqsjw&source=sh/x/aim/m1/4)

To build a **Data Retrieval Agent** using LangChain, you need to create a system that can understand a user's question, determine which data source or tool holds the answer, fetch that data, and synthesize a response.

- The step-by-step architecture to build one using **LangChain** and **LangGraph** (the modern standard for stateful agents).

Step 1: Define the LLM and Memory

First, initialize the central reasoning engine (the LLM) and a memory saver to track the conversation history.

- **LLM Setup:** Choose an execution model capable of reliable tool calling (e.g., GPT-4o, Claude 3.5 Sonnet, or Llama 3.1).
- **Memory:** Use MemorySaver to allow the agent to remember context across multi-turn dialogues.

Step 2: Build and Register Data Retrieval Tools

Tools are Python functions that your agent can execute to fetch external data. You must wrap them using the @tool decorator and provide clear docstrings so the LLM knows *when* and *how* to use them.

- **Vector Store / RAG Tool: For retrieving unstructured data (PDFs, internal wikis).**
- **SQL Database Tool: For querying structured transactional or user data.**
- **API / Web Search Tool: For pulling real-time data from external software or the internet.**

Step 3: Create the Agent State

Define what information the agent needs to track as it runs.

In LangGraph, this is typically a state object containing a list of messages.

Every time the LLM speaks or a tool runs, new messages are appended to this state.

Step 4: Construct the Agent Graph Nodes

An agent workflow consists of two primary operational nodes:

1. **The Agent Node: Takes the current state (conversation history), passes it to the LLM, and asks the LLM whether it needs to call a tool or reply directly to the user.**
2. **The Tool Execution Node: If the LLM decides to call a tool, this node intercepts the tool request, runs the Python function (e.g., executes an SQL query), and feeds the results back into the message history.**

Step 5: Define the Routing Logic & Edges

- **Link your nodes together with conditional logic:**
- **If the LLM's response contains tool_calls, route the graph to the Tool Execution Node.**
- **Once the tool finishes running, route back to the Agent Node so the LLM can read the retrieved data.**
- **If the LLM decides it has enough information to answer the user, route to the End of the graph.**

Step 6: Compile and Execute

Compile the graph workflow into a runnable application. When you invoke it with a user query, the agent will autonomously loop through steps 4 and 5 until it retrieves all necessary data to formulate its final answer.

Basic Code Architecture (Python)

```
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.memory import MemorySaver

# 1. Define Data Retrieval Tools
@tool
def query_company_database(customer_id: str) -> str:
    """Queries the SQL database for active customer subscription statuses."""
    # Your database fetching logic goes here
    return f"Customer {customer_id} is on the Premium Plan."

tools = [query_company_database]

# 2. Initialize the LLM with Tool Binding capabilities
llm = ChatOpenAI(model="gpt-4o", temperature=0)

# 3. Initialize Memory Checkpointer
memory = MemorySaver()

# 4. Compile the Agent Graph
agent_executor = create_react_agent(llm, tools, checkpointer=memory)

# 5. Run the Agent
config = {"configurable": {"thread_id": "user_session_1"}}
```

```
response = agent_executor.invoke(
    {"messages": [("user", "Check the database to see if customer CUST-99 is active.")]},
    config
)

print(response["messages"][-1].content)
```

---

3. https://www.analyticsvidhya.com/blog/2023/08/qa-applications/

Building Custom Q&A Applications Using LangChain and Pinecone Vector Database